

Chapter 01: Q-Learning

An Off-Policy, Model-Free Value-Based Algorithm

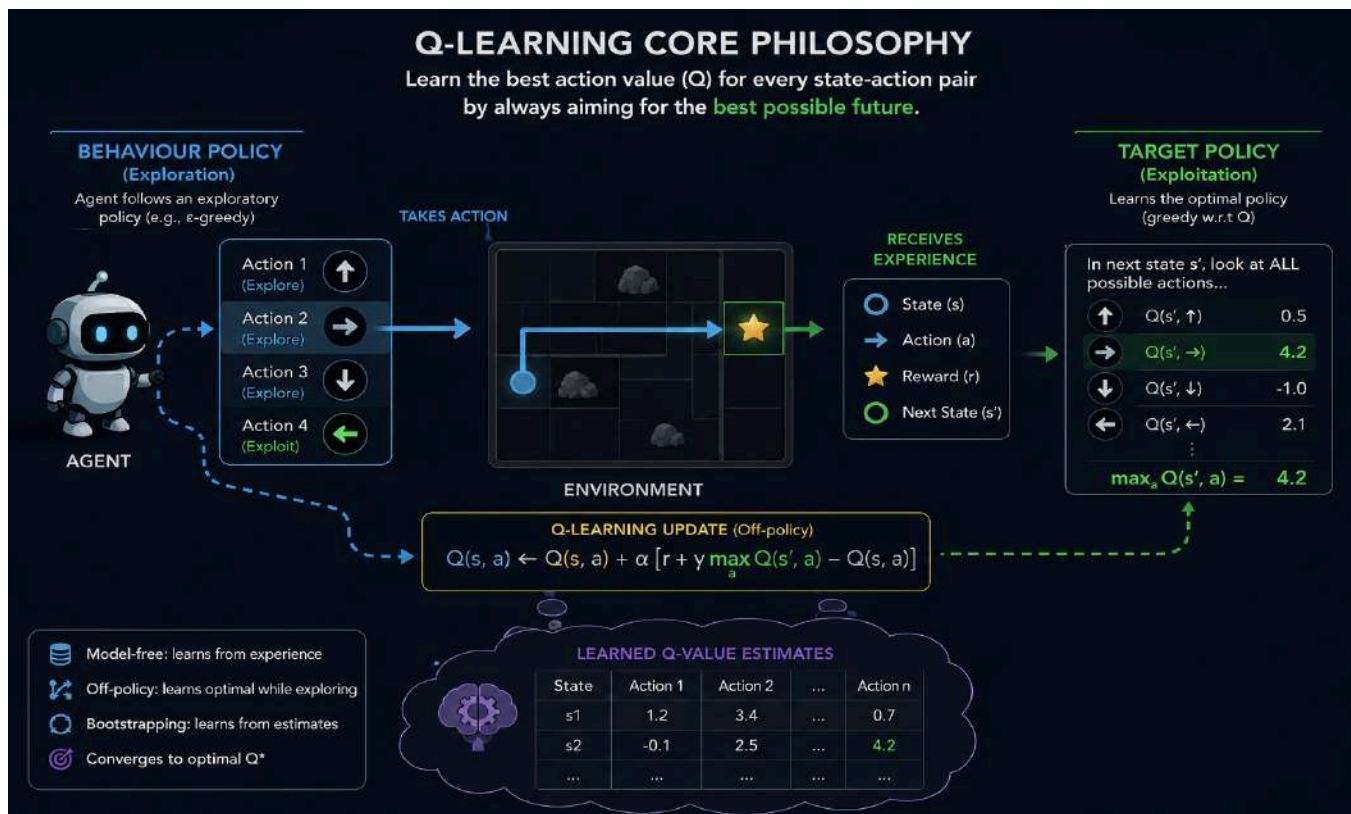
1.1 Introduction

Q-Learning is a model-free, off-policy reinforcement learning algorithm introduced by Christopher Watkins in 1989. It enables an agent to learn the value of taking a particular action in a particular state, without requiring any prior knowledge of the environment's transition dynamics or reward function. The algorithm learns an action-value function, commonly denoted $Q(s,a)$ which estimates the expected **cumulative future reward** obtainable by **taking action in states** and thereafter following an optimal policy.

Because Q-Learning is model-free, the agent does not need a model of the environment; it learns purely from experience gathered through interaction (trial and error). Because it is off-policy, the algorithm can learn the optimal policy even while following a different, more exploratory behaviour policy.

1.2 Core Philosophy

The central philosophy of Q-Learning is to separate the policy the agent follows while exploring the environment (the behaviour policy) from the policy it is trying to learn (the target policy). Q-Learning always updates its estimate of $Q(s,a)$ using the **maximum possible Q-value of the next state**, regardless of which action the agent actually takes next. This makes it inherently optimistic and goal-directed: it always assumes **the best possible future action** will be chosen, even if the agent's actual behaviour (e.g., an epsilon-greedy policy) sometimes explores sub-optimal actions.



- **Model-free:** learns directly from raw experience (**state, action, reward, next state**) tuples.
- **Off-policy:** the target policy (greedy, max-based) differs from the behaviour policy (e.g., ϵ -greedy).
- **Bootstrapping:** updates current estimates using other learned estimates (**Temporal-Difference learning**).
- **Convergence guarantee:** under suitable conditions (all state-action pairs visited infinitely often, decaying learning rate), Q-Learning provably converges to the optimal Q^* .

1.3 Working Process (Step-by-Step)

Key Components

What is a Q-table?

A Q-table is a **memory table** where the agent stores Q-values. It shows which action gives the best reward in each state. The agent updates it as it learns from experience, helping it make better decisions.

Structure of a Q-table:

- Rows represent the **states**.
- Columns represent the possible **actions**.
- Each entry in the table corresponds to the **Q-value** for a **state-action pair**.

Over time, the Q-table **improves** through **learning** and helps the agent choose the **best actions**.

Q-Values or Action-Values

Q-values represent the expected rewards for taking an action in a specific state. These values are updated over time using the **Temporal Difference (TD)** update rule.

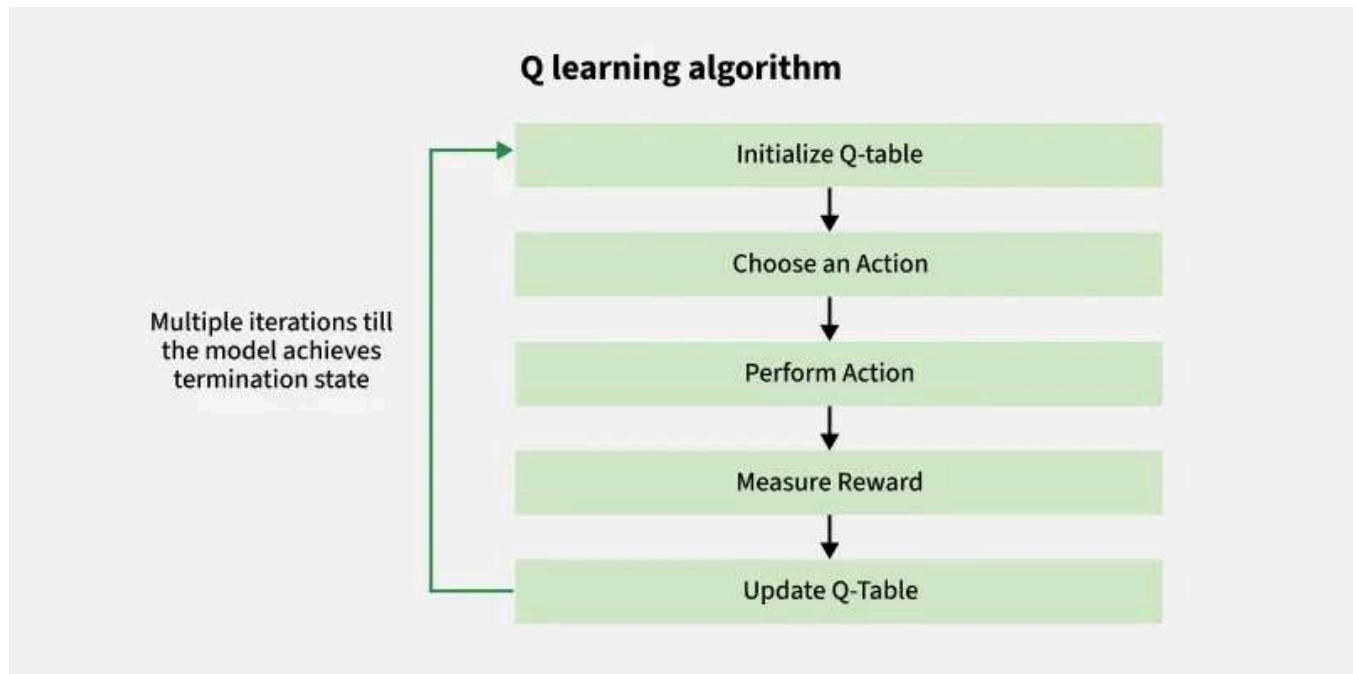
Rewards and Episodes

The agent moves through different states by taking actions and receiving rewards. The process continues until the agent reaches a terminal state which ends the episode.

The **Q-learning algorithm** allows the agent to learn by **exploring the environment** and **updating Q-values based on received rewards**. The **steps involved in the Q-learning algorithm process include**

- **Step 1** — Initialize a **Q-table** with one row per state and one column per action; typically initialized to zeros or small random values.
- **Step 2** — Observe the **current state s** .
- **Step 3** — Select an action using an exploration strategy such as **ϵ -greedy** (random action with probability ϵ , otherwise the action with the highest Q-value).
- **Step 4** — Execute **action a** , observe the **reward r** and the resulting **next state s'** .
- **Step 5** — **Update $Q(s, a)$ using the Q-Learning update rule**, which bootstraps from the maximum Q-value over all actions available in s' .

- **Step 6** — Set $s \leftarrow s'$ and repeat Steps 2–5 until the episode terminates (goal reached, failure state, or time limit).
- **Step 7** — Repeat over many episodes, gradually **decaying** ϵ so the agent shifts from exploration to exploitation, until the Q-values converge.



Epsilon-Greedy Action Selection Strategy

The **epsilon-greedy** (ϵ -greedy) strategy is one of the most widely used action-selection methods in reinforcement learning. Its primary purpose is to balance **exploration** (trying new or less-visited actions) and **exploitation** (selecting the action currently believed to be the best).

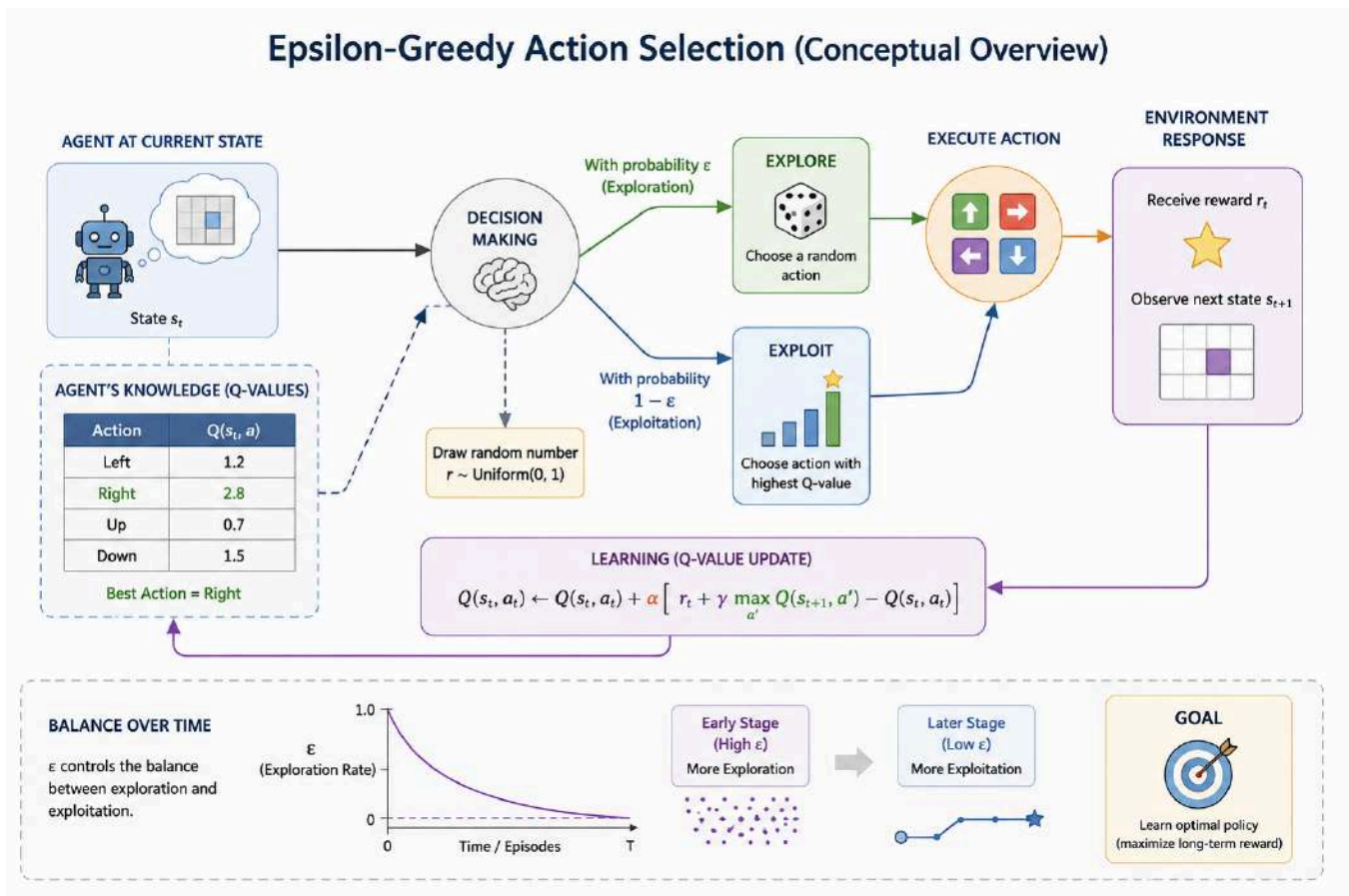
At every decision step, the agent chooses between:

- **Exploitation:** Selecting the action with the highest estimated Q-value with a high probability $(1 - \epsilon)$.
- **Exploration:** Selecting a random action with a small probability (ϵ) .

This balance enables the agent to discover potentially better actions while still taking advantage of the knowledge it has already acquired.

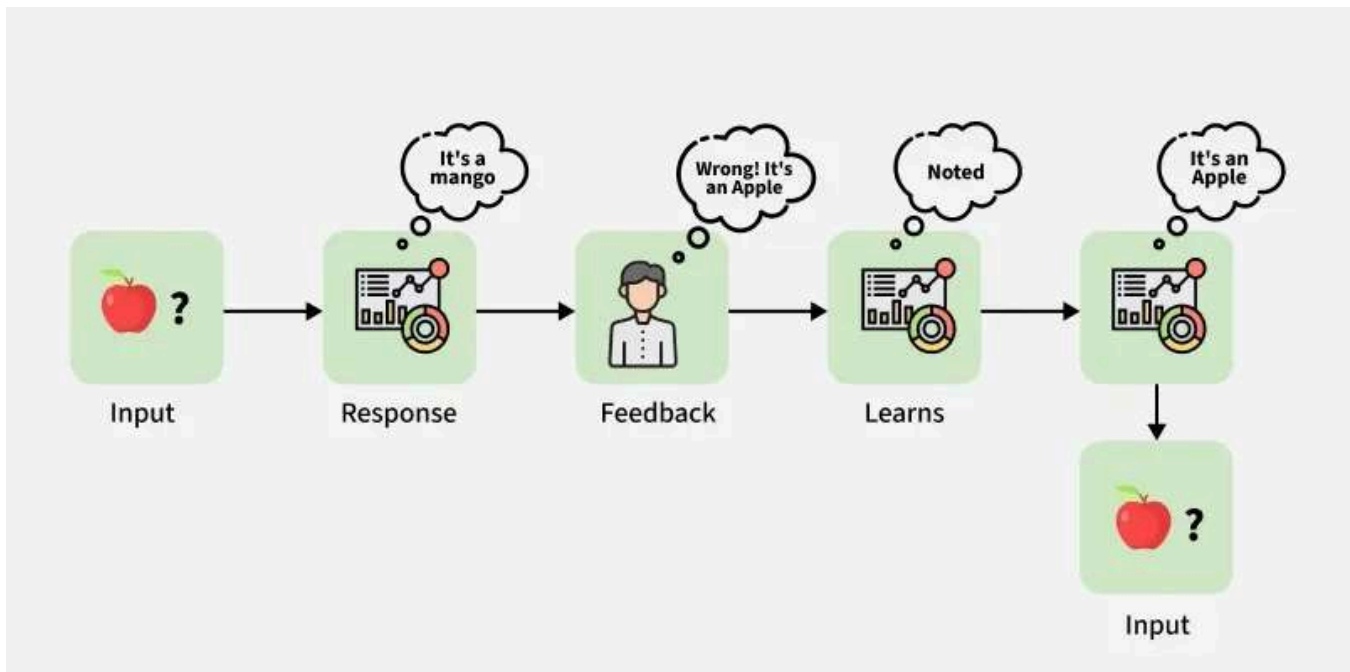
Without exploration, the agent may become trapped in a **local optimum**, repeatedly selecting the same action without discovering superior alternatives. By incorporating exploration, the epsilon-greedy strategy allows the agent to continue learning about the environment, particularly during the early stages of training. Consequently, it improves learning efficiency, robustness, and the ability to converge toward an optimal policy.

Epsilon-Greedy Action Selection (Conceptual Overview)



1.4 Simple Example

Imagine a system that sees **an apple** but incorrectly says, “It’s a mango.” The system is told, “Wrong! It’s an apple.” It learns from this mistake. **Next time, when shown the apple, it correctly says “It’s an apple.”** This **trial-and-error process**, guided by **feedback** is like how Q-Learning works.



1.5 Mathematical Formulation

The Q-Learning update rule (Temporal-Difference control) is:

$$Q(s, a) = Q(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

where:

- **α (alpha) — learning rate**, $0 < \alpha \leq 1$, controls how much new information overrides old estimates.
- **γ (gamma) — discount factor**, $0 \leq \gamma \leq 1$, controls how much future rewards are valued relative to immediate rewards.
- **r — immediate reward received** after taking action in state s .
- **$\max_{a'} Q(s', a')$ — the highest Q-value achievable from the next state s'** over all possible next actions a' (this is what makes it off-policy).
- The bracketed term $[r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$ is called the **TD-error** (temporal-difference error).

The optimal action-value function Q^* satisfies the Bellman optimality equation:

$$Q^*(s, a) = E[r + \gamma \cdot \max_{a'} Q^*(s', a') | s, a]$$

Here:

E = Expected value (average over all possible outcomes).

r = reward received.

s' = next state, which may be random.

Q^* = Optimal action-value function.

This equation works for both deterministic and stochastic environments.

1.6 Mathematical Simulation

Simulation Setup

Consider a discrete environment with four states $\{S_0, S_1, S_2, S_3\}$ and four actions {Left, Right, Up, Down}. The agent maintains a Q-table, initialised as shown in Table 1.

Table 1 — Initial Q-Table

State	Left	Right	Up	Down
S_0	0.0	1.5	0.0	0.0
S_1	2.0	3.0	1.0	0.5
S_2	1.2	2.0	0.8	1.5

State	Left	Right	Up	Down
S_3	4.5	8.0	6.5	5.0

The agent currently occupies state S_2 and selects the action Right.

Environment Response

Executing the action produces the transition tuple (s, a, r, s') , summarised in Table 2.

Table 2 — Transition Parameters

Parameter	Symbol	Value
Current state	s	S_2
Action taken	a	Right
Reward received	r	10
Next state	s'	S_3
Learning rate	α	0.5
Discount factor	γ	0.9
Current $Q(S_2, \text{Right})$	$Q(s, a)$	2.0
Max Q in next state	$\max_{a'} Q(S_3, a')$	8.0

Table 3 — Reward Function

Current State	Action	Next State	Reward (r)	Description
S_0	Right	S_1	-1	Normal movement
S_1	Right	S_2	-1	Normal movement
S_2	Right	S_3	+10	Successful progress toward the goal
S_2	Left	S_1	-5	Moving away from the goal
S_2	Up	S_2	-2	Invalid or ineffective move

S_2	Down	S_0	-3	Less desirable transition
S_3	Goal	Terminal	+100	Goal reached

Step-by-Step Q-Learning Update

Step 1 — Identify the target Q-value

The value to be updated is the current estimate for the executed state-action pair:

$$Q(S_2, \text{Right}) = 2.0$$

Step 2 — Locate the maximum Q-value in the next state

For state S_3 , the action-value vector across all four actions is:

$$Q(S_3, \cdot) = [4.5, 8.0, 6.5, 5.0]$$

Therefore:

$$\max_{a'} Q(S_3, a') = 8.0 \quad (\text{action} = \text{Right})$$

Step 3 — Compute the TD target

The temporal-difference (TD) target is the immediate reward plus the discounted value of the best available next action:

$$\begin{aligned}
 TD \text{ Target} &= r + \gamma \cdot \max_{a'} Q(s', a') \\
 TD \text{ Target} &= 10 + 0.9 \times 8.0 \\
 &= 10 + 7.2 \\
 &= 17.2
 \end{aligned}$$

Result — Step 3

TD Target = 17.2

Step 4 — Compute the TD error

The TD error (Bellman residual) is the gap between the TD target and the current estimate:

$$\begin{aligned}
 \delta &= TD \text{ Target} - Q(s, a) = 17.2 - 2.0 \\
 &= 15.2
 \end{aligned}$$

Result — Step 4

$$\bar{\delta} = 15.2$$

Step 5 — Apply the update rule

The new Q-value moves the old estimate a fraction α of the way toward the TD target:

$$\begin{aligned} Q_{new}(S_2, Right) &= Q(S_2, Right) + \alpha \cdot \delta \\ &= 2.0 + 0.5 \times 15.2 = 2.0 + 7.6 \end{aligned}$$

Result — Step 5 (Final Update)

$$Q_{new}(S_2, Right) = 9.6$$

Updated Q-Table

After a single update, only the cell corresponding to the experienced state-action pair changes; all other entries remain unchanged. This is the local update property of Q-learning.

Table 3 — Q-Table After Episode 1

State	Left	Right	Up	Down
S_0	0.0	1.5	0.0	0.0
S_1	2.0	3.0	1.0	0.5
S_2	1.2	9.6	0.8	1.5
S_3	4.5	8.0	6.5	5.0

Note: $Q(S_2, Right)$ moved from 2.0 to 9.6. The remaining fifteen cells are unaffected because the agent did not visit those state-action pairs during this transition.

Convergence Simulation Across Episodes

If the agent repeatedly experiences the same transition $(S_2, Right) \rightarrow S_3$ with reward $r = 10$, each subsequent update is computed exactly as above, using the previous episode's Q-value as the new starting point. Table 4 simulates six such episodes.

Table 4 — Convergence of $Q(S_2, \text{Right})$

Episode	$\delta = 17.2 - Q_{old}$	Update = $0.5 \cdot \delta$	Q_{old}	Q_{new}
1	15.20	7.60	2.00	9.60
2	7.60	3.80	9.60	13.40
3	3.80	1.90	13.40	15.30
4	1.90	0.95	15.30	16.25
5	0.95	0.475	16.25	16.73
6	0.47	0.235	16.73	16.96

The residual δ is halved at every step because $\alpha = 0.5$, so the sequence approaches its limit geometrically rather than linearly. The fixed point of this recursion is the TD target itself:

$$\lim_{n \rightarrow \infty} Q_n(S_2, \text{Right}) = r + \gamma \cdot \max_{a'} Q(S_2, a') = 10 + 0.9 \times 8.0 = 17.2$$

Convergence limit

$Q(S_2, \text{Right}) \rightarrow 17.2$ as episodes $\rightarrow \infty$, with the update magnitude shrinking by a factor of $(1 - \alpha) = 0.5$ at every visit.

General Closed-Form Expression

For a fixed target $T = r + \gamma \cdot \max_{a'} Q(s', a')$ and constant **learning rate** α , repeated updates to the same state-action pair follow a first-order linear recursion whose closed-form solution after n visits is:

$$Q_n = T - (T - Q_0) \cdot (1 - \alpha)^n$$

Substituting $T = 17.2$, $Q_0 = 2.0$, and $\alpha = 0.5$ reproduces every value in Table 4 exactly, confirming the geometric decay of the residual and the algorithm's guaranteed convergence to the TD target under a constant learning rate for this single state-action pair.

1.7 Algorithm: Pseudocode

Algorithm: Q-Learning

Initialize $Q(s, a)$ arbitrarily for all s in S , a in $A(s)$
Initialize $Q(\text{terminal_state}, \cdot) = 0$

for each episode:

 Initialize state s

 while s is not terminal:

 Choose action a from s using policy derived
 from Q (e.g., epsilon-greedy)

```

    Take action a, observe reward r and next state s'

    Q(s, a) <- Q(s, a) + alpha * [ r + gamma *
                                max_a' Q(s', a') - Q(s, a) ]

    s <- s'
  end while
end for

return Q

```

1.8 Python Implementation

Note: All practical implementations for this assignment are available in my Google Colab notebook.

[Click Here](#)

1.9 Advantages

- **Model-free:** does not require knowledge of transition probabilities or reward function in advance.
- **Off-policy learning** allows reuse of experience gathered under any exploratory behaviour policy.
- **Guaranteed convergence** to the optimal policy under standard stochastic approximation conditions.
- **Conceptually simple** and easy to implement for small, discrete state-action spaces.

1.10 Core Challenges & Limitations

- **Scalability:** the tabular Q-table grows exponentially with the number of states and actions, making it impractical for large or continuous state spaces.
- **Overestimation bias:** the max operator systematically overestimates action values, since it selects the maximum of noisy estimates.
- **Exploration–exploitation trade-off:** choosing an effective ϵ -decay schedule is non-trivial.
- **Slow convergence:** may require a very large number of episodes to converge in complex environments.

1.11 Real-World Applications

- **Robotics** — path planning and navigation in discretized grid or graph environments.
- **Game playing** — classic grid-world, maze, and simple board games.
- **Traffic signal control** — learning signal timing policies to minimize congestion.
- **Recommendation systems** — sequential decision-making for content or product recommendation.
- **Resource/inventory management** — dynamic allocation policies under uncertain demand.

Real-World Applications of Q-Learning

1. Robotics – Path Planning and Navigation



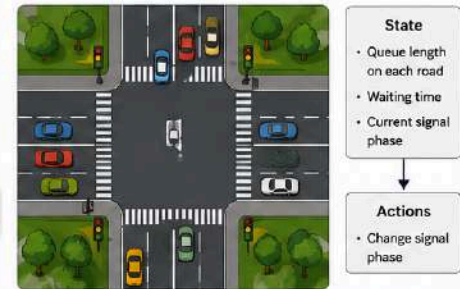
The robot learns the best path to reach the goal while avoiding obstacles.

2. Game Playing – Grid World / Maze



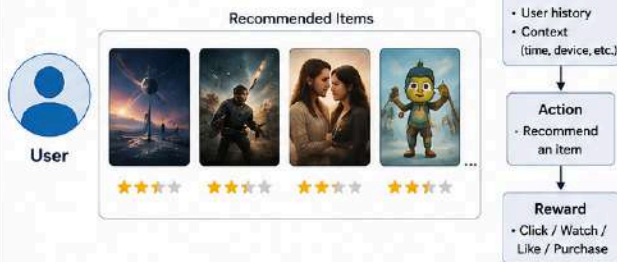
The agent learns which actions lead to the goal and maximizes total reward.

3. Traffic Signal Control



The system learns optimal signal timing policies to reduce total waiting time and congestion.

4. Recommendation Systems



The system learns what content a user is most likely to engage with to maximize long-term reward.

5. Resource / Inventory Management



The system learns how much inventory to order or how to allocate resources to maximize profit and minimize costs.

1.12 Future Directions

Research continues to address Q-Learning's overestimation bias (e.g., **Double Q-Learning**), improve sample efficiency through prioritized experience replay, and extend the tabular approach to function approximation for large or continuous spaces a direction that leads directly into **Deep Q-Networks**, discussed later in this report. Hybrid approaches combining Q-Learning with model-based planning, and safe/constrained variants for real-world deployment, remain active areas of study.

1.13 References for Further Reading

1. Watkins, C. J. C. H. (1989). *Learning from Delayed Rewards*. PhD Thesis, University of Cambridge.
2. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
3. Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3-4), 279–292.